

News Classification

End-to-End Multi-Class News Categorization Project — Build Guide

PROITBRIDGE · AI/ML COURSE · NLP PROJECT 3

About This Project

This guide walks through building a complete news classification system from scratch in Python. Starting from the Inshorts news dataset — thousands of short news items split across several CSV files — you set up a clean project structure, combine and load the data, clean each article's text using NLP techniques (lower-casing, punctuation removal, stop-word removal and lemmatization with spaCy), convert the text into numerical features with TF-IDF, then train and evaluate a Linear Support Vector Machine (Linear SVM) classifier that sorts every article into its news category — Politics, Sports, Technology, Business, and more.

By the end you will have a trained model, a command-line prediction tool, and a FastAPI web service exposing a `/predict` endpoint — the hand-off a news app, search index, or recommendation engine will later call — plus a routing step that files each classified article into its category feed. You build and train the model locally from this guide first; the news app and recommendation layer are built on top of it afterwards. The project is organized into the ten stages of the machine-learning lifecycle, each building on the last.

***What you'll build:** a project scaffold, a multi-file data-loading and preprocessing pipeline, a TF-IDF feature pipeline (with a CountVectorizer alternative), a trained Linear SVM news classifier, an evaluation report, an interactive prediction tool, a deployable REST API, and a routing step that feeds the search and recommendation layer you build later.*

Topics Coverage Map

Every concept from the NLP Foundations, Preprocessing & Vectorization syllabus and the stage of this guide where you put it into practice. TF-IDF is the primary vectorizer for this project (per the project architecture); CountVectorizer is covered as the simpler count-based alternative.

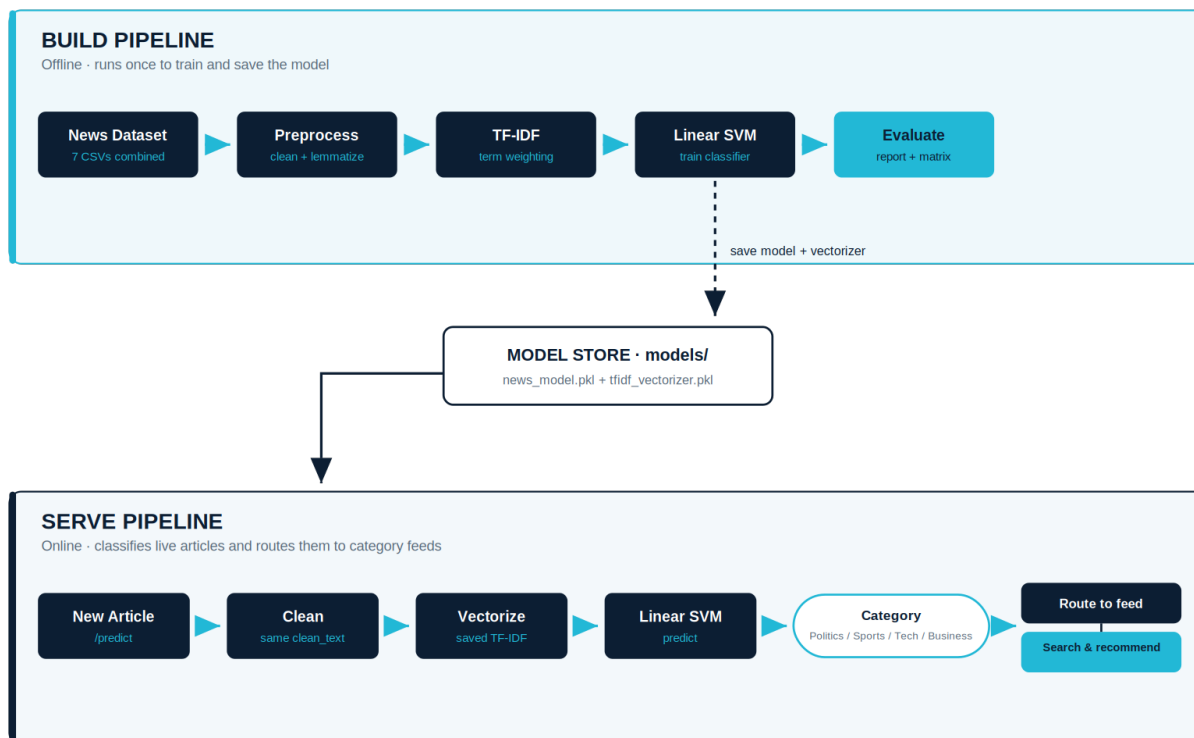
Topic	Where	How it's covered
Introduction to NLP	About, Stage 1	Overview of turning raw, messy news text into machine-ready numerical features.
NLP Pipeline	Stage 4	<code>process_dataset()</code> runs the full combine-clean-save pipeline over every CSV in one pass.
Text Cleaning & Lower Casing	Stage 4	<code>str(text).lower()</code> plus regex cleaning inside <code>clean_text()</code> normalize the text.
Punctuation & Stop Words	Stage 4	<code>re.sub(r"[^a-zA-Z]", ...)</code> strips punctuation; <code>token.is_stop</code> removes common stop-words.
Tokenization	Stage 4	<code>nlp(text)</code> runs spaCy, which splits each article into individual tokens.

Topic	Where	How it's covered
Stemming & Lemmatization	Stage 4	<code>token.lemma_</code> reduces each word to its base form (lemmatization, the more precise of the two).
Preprocessing Pipeline	Stage 4	<code>clean_text</code> and <code>clean_batch</code> applied to the whole combined dataset in one pass.
Corpus & Vocabulary	Stage 4	<code>TfidfVectorizer</code> learns the vocabulary from the cleaned news corpus.
N-Grams	Stage 4	<code>ngram_range=(1,2)</code> captures both single words and two-word phrases as features.
Bag of Words	Concept (Stage 4)	The plain-count model; covered as the <code>CountVectorizer</code> alternative to TF-IDF.
Count Vectorizer	Stage 4	The count-based alternative — raw word counts instead of TF-IDF weights.
TF · IDF · TF-IDF	Stage 4	The core vectorizer for this project; weights words by how distinctive they are across articles.
Linear SVM	Stage 5	<code>LinearSVC</code> — the large-margin classifier that excels on high-dimensional sparse TF-IDF features.
Multi-Class Classification	Stages 5–8	One model sorts each article into one of many categories, not just two.

System Architecture

SYSTEM DESIGN · HOW THE PIECES FIT

Before building anything, here is the shape of the whole system. It is two pipelines that meet at one shared model store. The build pipeline runs offline and turns the combined news dataset into a trained model saved on disk. The serve pipeline runs online: it loads that saved model to classify live articles, routes each one to its category feed, and records every classified article so a search index or recommendation engine can surface it.



End-to-end architecture — the offline build pipeline (top) trains and saves the model; the online serve pipeline (bottom) loads it to classify incoming articles and route them to category feeds for search and recommendations.

Two pipelines, one model store

The build pipeline is the data scientist's work (Stages 1–8). Articles are loaded from every CSV with their category labels and cleaned with spaCy; the cleaned text is vectorized with TF-IDF (or CountVectorizer); a Linear SVM model is trained on a class-balanced training set and then evaluated. The trained model and the fitted vectorizer are then saved into the models/ folder — the single source of truth the rest of the system depends on.

The serve pipeline is the online path (Stage 9 onward). A new article arrives at the FastAPI /predict endpoint, is cleaned by the exact same clean_text used in training, transformed by the saved vectorizer, and scored by the saved model. The predicted category is returned as JSON, and route_article (Stage 9) files the article into that category's feed — the store the search index and recommendation engine, the layer built on top of this model, later read.

Note: Both pipelines must clean and vectorize text the same way — a model only makes sense on features built exactly as it was trained. The model store is what guarantees that: the serve pipeline never re-derives anything, it loads the very vectorizer and model the build pipeline saved.

How the code is organised

Each box above maps to a folder in the project, so the architecture and the repository line up one to one:

- **src/data/** — loading every CSV and spaCy cleaning (Stages 3–4).

- **src/features/** — the TF-IDF and optional CountVectorizer vectorizers, built in Data Preparation (Stage 4).
- **src/models/** — building, training, evaluation, and the interactive predictor (Stages 5–8).
- **src/api/** — the FastAPI service that exposes /predict (Stage 9).
- **src/routing/** — the category-feed store the search and recommendation layer reads (Stage 9).
- **models/** — the saved artifacts both pipelines share.
- **data/** — the raw source CSVs and the processed, cleaned dataset.

The data flow, one line each:

```
Training: inshort_news_data-*.csv -> combine -> clean -> vectorize -> train -> models/
```

```
Serving: article -> /predict -> clean -> vectorize -> model -> category
         -> route_article -> category feed -> search & recommendations
```

Stage 1: Business Understanding

Every project starts here: before a line of code, get clear on the problem and on how success will actually be delivered. News publishers and aggregators receive thousands of articles every day from wire services, reporters, and partner feeds. Sorting each one into the right section — Politics, Sports, Technology, Business — by hand does not scale, and inconsistent tagging makes articles hard to find and weakens every recommendation built on top of them.

The goal: automatically classify each incoming news article into its category using NLP and Machine Learning, route it to the right feed, and power search and recommendations — improving how quickly stories reach the right readers across the platform.

Why not just do it by hand or with fixed rules?

The slide for this project lays out exactly why a manual or rule-list approach falls short, and why an NLP model is the right tool. The two columns below are the heart of the business case.

Without NLP (the old way)	With NLP (this project)
Editors tag each article by hand	Articles auto-sorted: Politics / Sports / Tech / Business
Categories are slow and inconsistent	Consistent labels at scale
Can't scale to thousands of articles	Instant routing as news breaks
Readers get weak recommendations	Powers search & recommendations

Manual tagging does not scale — nobody can read every article — and hand-written rule-lists (“file anything with the word ‘election’ under Politics”) are brittle: a sports story about a team electing its captain lands in the wrong section, and the rules quietly go stale as topics shift. An NLP model instead learns the statistical fingerprint of each category from real articles, so it generalizes to wording it has never seen and can be retrained as the news evolves.

How the finished system works, end to end:

- A new article arrives — typed into the prediction tool, or sent to the /predict API.
- The text is cleaned exactly the way the model was trained to expect (Stage 4).
- The model classifies it into one category — Politics, Sports, Technology, Business, and so on (Stages 5–6).
- The article is routed to that category’s feed (Stage 9) rather than left untagged.
- A search index and recommendation engine (the layer built later) read those feeds, so the right stories surface for the right readers.

This framing drives later choices — which mistakes matter most in Stage 8 (a Politics story filed under Business is a mild slip; a breaking story sent to the wrong feed entirely is worse), how we keep small categories from being ignored by balancing the training data in Stage 6, and the routing hand-off in Stage 9. The scope of this guide is to build and train the model locally; the news app, search index, and recommendation layer are built on top of the /predict API afterwards.

Stage 2: Data Collection

We gather everything the project needs before touching the data: a Python environment with the right libraries, a clean folder structure, and the dataset itself.

Set Up the Project

SETUP · ENVIRONMENT

Create the project folder.

Why: A dedicated folder keeps every file for this project in one isolated, organized place.

```
mkdir news_classification
cd news_classification
```

Create virtual environment.

Mac/Linux

Why: A virtual environment isolates this project’s libraries so they don’t clash with other Python projects on your machine.

```
python3 -m venv .venv
```

Activate:

Why: Activating makes every install and run command use this project’s isolated Python instead of the system one.

```
source .venv/bin/activate
```

Windows

Why: Same isolation step for Windows — create the environment, then activate it.

```
python -m venv .venv
.venv\Scripts\activate
```

Install Libraries

Create:

Why: Listing dependencies in one file lets anyone recreate the exact same environment with a single command.

```
requirements.txt
```

Paste:

Why: These are the libraries the project relies on — data handling (pandas, numpy), machine learning (scikit-learn), NLP (nltk, spacy), the web API (fastapi, uvicorn), plotting (matplotlib, seaborn), and resampling (imbalanced-learn).

```
pandas
numpy
scikit-learn
nltk
spacy
regex
joblib
fastapi
uvicorn
matplotlib
seaborn
imbalanced-learn
requests
```

Install:

Why: This installs everything listed in requirements.txt in one go.

```
pip install -r requirements.txt
```

Install SpaCy model:

Why: spaCy needs its language model downloaded separately; this English model powers the tokenizing and lemmatization used later.

```
python -m spacy download en_core_web_sm
```

Create the Folder Structure

SETUP · PROJECT STRUCTURE

Inside project:

Why: *A clear layout separates data, models, notebooks, and source code so the project stays maintainable as it grows.*

```
news_classification/
```

```
├── data/
│   ├── raw/
│   ├── processed/
│   └── feeds/
├── models/
├── notebooks/
├── src/
│   ├── api/
│   ├── data/
│   ├── features/
│   ├── models/
│   ├── routing/
│   └── utils/
├── requirements.txt
├── README.md
└── main.py
```

Create them manually or:

Why: *These commands build the whole structure automatically instead of creating each folder by hand.*

```
mkdir -p data/raw
mkdir -p data/processed
mkdir -p data/feeds

mkdir -p src/api
mkdir -p src/data
mkdir -p src/features
mkdir -p src/models
mkdir -p src/routing
mkdir -p src/utils

mkdir models
mkdir notebooks
```

Add the Dataset

Put the dataset files here:

Why: *Keeping the untouched originals in a dedicated raw folder means we always have the source data to fall back to.*

```
data/raw/inshort_news_data-1.csv
data/raw/inshort_news_data-2.csv
data/raw/inshort_news_data-3.csv
data/raw/inshort_news_data-4.csv
data/raw/inshort_news_data-5.csv
data/raw/inshort_news_data-6.csv
data/raw/inshort_news_data-7.csv
```

The dataset arrives as several CSV files that share the same three columns, one row per article:

```
news_headline,news_article,news_category
50-year-old problem of biology solved by AI,"DeepMind's AlphaFold predicts how proteins
fold...",technology
Global smartphone sales in Q3 fall 5.7%,"Global smartphone sales fell 5.7% year-on-
year...",technology
Ian Chappell urges ICC to ban 'switch hit',"Ex-Australia captain Ian Chappell has
urged...",sports
```

Note: Unlike a sentiment project where labels must be derived from the text, this dataset is already labeled — every article comes with a `news_category`. That is the target the model learns to predict. The only extra step versus a single-file project is combining the seven CSVs into one dataset, which we handle in Stage 3.

Stage 3: Data Understanding

Create file:

***Why:** We keep the data-loading logic in its own module so it can be reused and tested independently.*

```
src/data/load_data.py
```

Paste:

***Why:** The dataset is split across several CSVs, so this function globs every `inshort_news_data-*.csv` in the folder and concatenates them into one pandas `DataFrame` — the starting point for everything that follows.*

```
# Lifecycle stage 3 – Data Understanding

import glob
import pandas as pd

def load_data(folder):

    files = glob.glob(f"{folder}/inshort_news_data-*.csv")

    frames = [pd.read_csv(f) for f in files]

    return pd.concat(frames, ignore_index=True)
```


Test It

Create:

Why: main.py is the entry point we use to run and verify each piece as we build it.

```
main.py
```

Why: Loading the data and printing the shape, columns, and category counts confirms every CSV was found and combined, and shows how the categories are spread before moving on.

```
from src.data.load_data import load_data

df = load_data("data/raw")

print(df.shape)
print(df.columns)
print(df["news_category"].value_counts())
```

Run:

```
python main.py
```

Expected:

```
(several thousand, 3)

Index(['news_headline', 'news_article', 'news_category'], dtype='object')

technology    ...
sports        ...
business      ...
...
```

You should see the combined row count, the three column names, and how many articles fall in each category.

Note: Real category counts are rarely even — technology and sports often dominate while categories like science or automobile are much smaller. That imbalance is normal; Stage 6 (Model Training) handles it by balancing the training data so the rare categories are not ignored.

Stage 4: Data Preparation

We turn raw articles into model-ready inputs. Because the dataset already carries a `news_category` label, there is no labeling step — the work here is combining the headline and article text, cleaning it, applying that across the whole dataset, and finally extracting numerical features (vectorization). All of this is data preparation; the model itself comes later.

Clean the Article Text

Create:

Why: Raw news text is messy — mixed case, punctuation, quotation marks, numbers, and filler words — so we clean it to help the model focus on the words that actually separate one category from another.

```
src/data/preprocess.py
```

Paste:

Why: This function lowercases the text, strips URLs and non-letters, removes stop-words, and reduces each word to its base form (lemma). For news, words like “election”, “goal”, “smartphone”, and “revenue” are strong category signals, and lemmatization makes sure “wins”, “winning”, and “won” all collapse to the same feature.

```
# Lifecycle stage 4 – Data Preparation (text cleaning)

import re
import spacy

nlp = spacy.load("en_core_web_sm")

def clean_text(text):

    text = str(text).lower()

    text = re.sub(r"http\S+", "", text)

    text = re.sub(r"[^a-zA-Z ]", "", text)

    doc = nlp(text)

    words = []

    for token in doc:

        if not token.is_stop:
            words.append(token.lemma_)

    return " ".join(words)

def clean_batch(texts):

    cleaned = []

    # nlp.pipe streams the texts through spaCy in batches;
    # disabling parser + ner skips the slow components we don't use

    docs = nlp.pipe(texts,
```

```

        disable=["parser", "ner"],
        batch_size=200)

    for doc in docs:

        words = [t.lemma_ for t in doc if not t.is_stop]

        cleaned.append(" ".join(words))

    return cleaned

```

Why two functions: *clean_text* cleans one article at a time (used by the prediction tool and the API). *clean_batch* does the same work but streams many articles through spaCy in batches and skips the parser and NER components — this is what keeps processing fast across a dataset of thousands of articles. We use *clean_batch* for the full dataset in Data Preparation.

Note: If you ever change *clean_text* after already building the project, you must re-run the Data Preparation, Model Building, and Model Training stages. The saved vocabulary and model are baked from the old cleaning, so predictions stay wrong until you rebuild them.

Test Cleaning

Update main.py

Why: *Running it on a sample headline confirms the cleaning produces tidy, normalized output before we apply it to the whole dataset.*

```

from src.data.preprocess import clean_text

sample = """Global smartphone sales in Q3 fall 5.7% to 36.6 cr units: Gartner"""

print(clean_text(sample))

```

Run:

```
python main.py
```

Expected:

```
global smartphone sale fall cr unit gartner
```

Punctuation, the “5.7%”, and the stop-words are gone; the category-signal words remain.

Process the Entire Dataset

Create:

Why: *This ties the previous steps together — combining every CSV, joining the headline and article into one text field, cleaning the whole dataset in one pass, and saving the result so we only do this work once.*

```
src/data/process_dataset.py
```

Why: Wrapping it in a function with print checkpoints makes the pipeline reusable and easy to follow as it runs. We join headline + article, do the fast lowercasing and regex as vectorized pandas operations, then hand only the tokenizing and lemmatizing to spaCy via clean_batch.

```
# Lifecycle stage 4 – Data Preparation (full dataset)

import pandas as pd
from src.data.load_data import load_data
from src.data.preprocess import clean_batch

def process_dataset():

    print("Script Started")

    df = load_data("data/raw")

    print("Dataset Loaded")

    # Headline + article together give the model more signal than either alone.
    combined = df["news_headline"].astype(str) + " " + df["news_article"].astype(str)

    # Lowercasing + regex are fast as vectorized pandas ops,
    # so we do them here and leave only tokenizing/lemmatizing to spaCy.
    texts = combined.str.lower()
    texts = texts.str.replace(r"http\S+", "", regex=True)
    texts = texts.str.replace(r"^[a-zA-Z ]", "", regex=True)

    df["clean_text"] = clean_batch(texts)

    print("Cleaning Completed")

    df.to_csv("data/processed/cleaned_news.csv", index=False)

    print("File Saved")
```

Update main.py

Why: Calling the function from main.py runs the full cleaning pipeline end to end.

```
from src.data.process_dataset import process_dataset

process_dataset()
```

Run:

```
python main.py
```

Output:

```
Script Started
```

```
Dataset Loaded  
Cleaning Completed  
File Saved
```

Check:

Why: Saving the cleaned data means later stages can load it instantly instead of re-running the cleaning every time.

```
data/processed/cleaned_news.csv
```

created successfully, with a new `clean_text` column next to the original `news_headline`, `news_article`, and `news_category`.

A Note on Class Balance

Why: Before feeding the data to a model, we always check how the categories are spread. One line tells us:

```
print(df["news_category"].value_counts())
```

Output (illustrative):

technology	7000
sports	5500
business	3200
world	2100
automobile	900
science	600

News categories are usually uneven — a few dominate, others are small. If the spread is extreme, the model can lean toward the big categories and neglect the rare ones. We only diagnose it here; the fix lives inside Stage 6 (Model Training), where the model is fitted.

Note: Don't rebalance the data now. The balancing has to happen after the train/test split, on the training fold only — if you duplicate rows before splitting, copies leak into the test set and your scores look better than the model really is. That is exactly why the fix belongs in Stage 6 (split first, then oversample the training data), not here in Data Preparation.

Vectorize the Text (Feature Extraction)

The final data-preparation step turns the cleaned text into numbers. This is feature extraction — it produces the numerical features the model will later be built and trained on, so it belongs to Data Preparation, not Model Building. No model is involved yet; we are only shaping the inputs. This project uses TF-IDF, with `CountVectorizer` as the simpler count-based alternative.

Create:

Why: TF-IDF turns each article into a vector where every word is weighted by how distinctive it is. A word like “the” appears everywhere and gets near-zero weight, while “smartphone” or “parliament” — words that show up in some categories far more than others — get boosted.

That weighting is exactly what helps a linear classifier draw clean boundaries between categories.

```
src/features/vectorizer.py
```

Paste:

Why: *This fits the vectorizer on the cleaned articles and saves it as a reusable artifact, so the exact same transformation can be loaded at training and prediction time. `max_features` caps the vocabulary to the most informative words; `gram_range=(1,2)` also captures two-word phrases like “prime minister” or “stock market”.*

```
# Lifecycle stage 4 – Data Preparation (feature extraction)

import pandas as pd
import joblib

from sklearn.feature_extraction.text import TfidfVectorizer

df = pd.read_csv("data/processed/cleaned_news.csv")
df["clean_text"] = df["clean_text"].fillna("")

vectorizer = TfidfVectorizer(max_features=20000, ngram_range=(1, 2))

X = vectorizer.fit_transform(df["clean_text"])

joblib.dump(vectorizer, "models/tfidf_vectorizer.pkl")

print(X.shape)
```

Run:

```
python src/features/vectorizer.py
```

Output:

```
(25000, 20000)
```

Example — 25,000 articles, each turned into a row of TF-IDF weights across the learned vocabulary (your exact counts will differ).

Check:

```
models/
    tfidf_vectorizer.pkl
```

TF-IDF vs CountVectorizer — which to use

Both turn text into numbers; they differ in how they weight words:

- **TF-IDF** — counts re-weighted so words common to every article are played down and rarer, more distinctive words are boosted. It pairs especially well with a linear classifier like Linear SVM, which is why it is the default for this project.
- **CountVectorizer** — plain word counts (Bag of Words). Simpler and a natural fit for Naive Bayes, but on a multi-category problem with a large vocabulary the raw counts let very common words dominate.

To try CountVectorizer instead, swap the import and class — everything else stays the same:

```
from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer(max_features=20000, ngram_range=(1, 2))
```

Note: Keep one vectorizer live at a time and always pair it with a model trained on it. A model trained on TF-IDF features will give meaningless predictions on CountVectorizer features, and vice-versa — the serve pipeline must load the very vectorizer the model was trained with.

Stage 5: Model Building

With the features prepared in Stage 4, Model Building is where we choose and configure the algorithm that will learn from them. Note the boundary: turning text into numbers was feature extraction (Stage 4); here we only select and set up the model. Nothing is fitted yet — that happens in Stage 6.

For news classification we build a Linear Support Vector Machine (LinearSVC).

***Why Linear SVM for text:** it draws the widest possible boundary between categories in the very high-dimensional space TF-IDF produces, where each article is a sparse vector over tens of thousands of words. It is fast to train, handles that dimensionality without strain, and is one of the strongest baselines for multi-class text classification — which is exactly why the project architecture pairs it with TF-IDF.*

Define the Model

Create:

***Why:** Keeping the model definition in its own module means the algorithm and its settings live in one place, so training, evaluation, and any later retraining all build the exact same model.*

```
src/models/build_model.py
```

Paste:

***Why:** build_model() returns a fresh, untrained Linear SVM. C controls the trade-off between a wide margin and classifying every training point correctly; the default 1.0 is a sound starting point. scikit-learn handles the multi-class case automatically with a one-vs-rest scheme, so one LinearSVC covers all categories. Stage 6 calls this to get the model, then fits it.*

```
# Lifecycle stage 5 – Model Building
```

```
from sklearn.svm import LinearSVC

def build_model():
    # C = regularization strength; one-vs-rest handles all categories
    return LinearSVC(C=1.0)
```

Note: Model Building is also where you would swap in or compare other classifiers — for example `LogisticRegression(max_iter=1000)` or `MultinomialNB()` — by returning a different model from `build_model()`. Because the features are prepared separately in Stage 4, only this one function changes; the rest of the pipeline stays the same.

Stage 6: Model Training

This is where the model defined in Stage 5 is actually fitted to the features prepared in Stage 4, then saved. It is also where we deal with class imbalance — not as a separate stage later, but right here, as part of the normal training flow. The order is deliberate: load the data, split it, balance the training half, fit, save.

Create:

```
src/models/train.py
```

***Why:** We load the cleaned data and the vectorizer saved in Stage 4 (we transform with it, never re-fit — fitting was a data-preparation step), build the untrained model from Stage 5, split into train and test sets, balance the training set so no category is starved of examples, fit the model, and save it. `stratify=y` keeps each category's proportion the same in both splits before balancing.*

```
# Lifecycle stage 6 – Model Training

import pandas as pd
import joblib
from collections import Counter

from sklearn.model_selection import train_test_split
from imblearn.over_sampling import RandomOverSampler
from src.models.build_model import build_model

df = pd.read_csv("data/processed/cleaned_news.csv")
df["clean_text"] = df["clean_text"].fillna("")

# Load the vectorizer prepared in Stage 4 – transform only, never re-fit.
vectorizer = joblib.load("models/tfidf_vectorizer.pkl")

X = vectorizer.transform(df["clean_text"])
y = df["news_category"]
```



```
# Split FIRST – so balancing can never touch the test set.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Balance the TRAINING split only: duplicate rows from the smaller
# categories until every category has as many as the largest one.
print("Before balancing:", Counter(y_train))

ros = RandomOverSampler(random_state=42)
X_train, y_train = ros.fit_resample(X_train, y_train)

print("After balancing: ", Counter(y_train))

# Build the untrained model from Stage 5, then fit it on the balanced data.
model = build_model()

model.fit(X_train, y_train)

joblib.dump(model, "models/news_model.pkl")

print("Training Completed")
```

Run Training:

```
python src/models/train.py
```

Output (illustrative):

```
Before balancing: Counter({'technology': 5600, 'sports': 4400, 'business': 2400,
'politics': 1800})
After balancing: Counter({'technology': 5600, 'sports': 5600, 'business': 5600,
'politics': 5600})
Training Completed
```

Note: The vectorizer is loaded here, not created — it was fitted and saved during Data Preparation (Stage 4). Training only fits the model. This keeps a clean line between preparing features and building/training the model, and guarantees the serve pipeline later uses the very same vectorizer.

How the balancing works: RandomOverSampler

The three lines that balance the data are the heart of this stage, so it is worth understanding exactly what they do. `RandomOverSampler` comes from the `imbalanced-learn` library (installed back in Stage 2 as `imbalanced-learn`, imported as `imblearn`). It is a resampling tool: it changes the make-up of the training set so the model sees the categories evenly.

The technique is random oversampling. It looks at how many articles each category has, finds the largest category, and then randomly copies articles from every smaller category — sampling with replacement — until each one reaches that same count. In the output above, `technology` already had the most rows (5,600), so `business`, `sports`, and `politics` are each oversampled up to

5,600. `fit_resample` returns the rebalanced `X_train` and `y_train`, which we assign straight back over the originals before fitting.

Why: *A model trained on lopsided data can score high overall accuracy while quietly mislabeling the rare categories — it learns it can be “right most of the time” just by leaning toward the big categories. Balancing removes that shortcut: with every category equally represented, the model has to actually learn what makes a business article a business article, not just bet on the popular labels.*

One detail matters for text specifically. `RandomOverSampler` duplicates real articles — it never invents new ones. That is the right choice here because the features are 20,000-dimension sparse TF-IDF vectors. A technique like SMOTE, which builds synthetic rows by interpolating between existing vectors, would produce points halfway between two articles that do not correspond to any real article and can blur the line between close categories. Random oversampling keeps every training vector a genuine article’s fingerprint, so the model never learns from text that was never written.

Note: Balance the TRAINING split only, never the test set — which is why the split comes first in the code. If you oversample before `train_test_split`, duplicated articles land in both halves, the model is then “tested” on rows it already memorized, and your scores look far better than the model really is. Always split first, then resample `X_train` / `y_train`.

Note: The trade-off of oversampling is mild overfitting: because the model sees the same rare-category articles several times, it can lean a little too hard on those exact examples. For this project it is a safe, simple default. If you ever want a no-duplication alternative, `LinearSVC` also accepts `class_weight="balanced"`, which makes the model penalise mistakes on rare categories more heavily without changing the data at all — set it inside `build_model()` instead of oversampling here.

Seeing the effect: with and without balancing

The clearest way to teach why this line matters is to train the model both ways and compare. Run `train.py` once with the `RandomOverSampler` line commented out, note the result, then put it back and run again. The metric to watch is the per-category recall in the evaluation report (Stage 8) — not the overall accuracy, which hides the problem.

Why: *Overall accuracy can stay high while a small category quietly collapses: the model scores well on the plentiful categories, and the rare one barely moves the average. Per-category recall is what reveals whether a category is actually being learned or just ignored.*

Without balancing (illustrative, skewed data):

	precision	recall	f1-score
business	0.91	0.42	0.57
politics	0.88	0.55	0.68
sports	0.97	0.99	0.98
technology	0.93	0.99	0.96
accuracy			0.92

Accuracy reads a healthy 0.92, yet business recall is only 0.42 — the model misses more than half of all business articles, leaking them into other feeds. The headline number hides a category that is barely working.

With balancing (same model, RandomOverSampler on):

	precision	recall	f1-score
business	0.84	0.83	0.83
politics	0.86	0.85	0.85
sports	0.98	0.98	0.98
technology	0.96	0.96	0.96
accuracy			0.93

Overall accuracy barely moved (0.92 → 0.93), but business recall jumped from 0.42 to 0.83 and politics from 0.55 to 0.85. Same model, same features — only the training data was rebalanced — and the small categories are now genuinely learned. That gap, and how balancing closes it, is the whole concept to teach.

Note: How large this gain is depends entirely on how skewed the data is. On a near-balanced dataset the two reports look almost identical and balancing earns little; on a strongly skewed one the rare-category recall can swing sharply, as above. So the real lesson is the method for deciding: read per-category recall first, then balance only if a category is being left behind.

How this works in a production pipeline

A fair question from learners: is this balancing just a teaching trick, or is it what actually runs in production? The answer is that `train.py` is the production training script. The `RandomOverSampler` line goes in exactly the same place — after the split, before the fit — whether you run the script by hand today or an automated job runs it next month. Nothing is bolted on as a separate phase later; balancing is simply part of how the model is trained.

Why: Keeping balancing inside `train.py`, rather than in a one-off optimization script, means it can never be forgotten. Every retrain — by you now, or by the scheduler later — runs the same split-then-balance-then-fit sequence, so the model stays balanced automatically without anyone remembering to repeat a separate step.

In a real deployment, the only things that change around this line are the trigger and a switch:

- **The trigger** — today you run `python src/models/train.py` yourself. In production, the CI/CD pipeline (Stage 10) runs that same command on a schedule, or whenever enough fresh articles have accumulated.
- **A config switch** — instead of commenting the line in and out, production code reads a flag, so the same script can retrain with or without balancing per run, with no code edits.

That switch looks like this — the balancing still sits in the identical spot, just guarded by a flag:

```
# Read from a config file, an environment variable, or a CLI argument.
BALANCE = True

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

```

if BALANCE:
    ros = RandomOverSampler(random_state=42)
    X_train, y_train = ros.fit_resample(X_train, y_train)

model = build_model()
model.fit(X_train, y_train)

```

Note: Two rules carry straight from teaching into production unchanged: balance the training split only (never the test set), and always split before you resample. The Stage 8 evaluation and the Stage 10 retrain checks both run on the untouched test set, so the metrics that decide whether a new model goes live stay honest — balancing helps the model learn, it never inflates the score it is judged by.

Stage 7: Model Testing

Create:

***Why:** This gives a quick, interactive way to try the trained model on your own typed headlines or articles.*

```
src/models/predict.py
```

***Why:** It loads the saved model and vectorizer, then loops so you can keep entering article text and instantly see the predicted category. Each input is passed through `clean_text` first, so it is processed the exact same way as the training data — without this, the model sees a different kind of text than it learned from and predictions become unreliable.*

```

# Lifecycle stage 7 – Model Testing

import joblib
from src.data.preprocess import clean_text

model = joblib.load("models/news_model.pkl")

vectorizer = joblib.load("models/tfidf_vectorizer.pkl")

while True:

    text = input("\nEnter News: ")

    X = vectorizer.transform([clean_text(text)])

    prediction = model.predict(X)

    print("\nPrediction:", prediction[0])

```

Run:

```
python -m src.models.predict
```

Test:

Enter News:
India beat Australia by six wickets to win the series

Prediction:
sports

Stage 8: Model Evaluation

Create:

Why: Before trusting the model, we measure how accurately it predicts on data it never saw during training.

```
src/models/evaluate.py
```

Why: Loading the saved model and vectorizer and scoring the held-out test set tells us how well it really performs, category by category.

```
# Lifecycle stage 8 – Model Evaluation

import pandas as pd
import joblib

from sklearn.metrics import classification_report, confusion_matrix
from sklearn.model_selection import train_test_split

df = pd.read_csv("data/processed/cleaned_news.csv")
df["clean_text"] = df["clean_text"].fillna("")

model = joblib.load("models/news_model.pkl")

vectorizer = joblib.load("models/tfidf_vectorizer.pkl")

X = vectorizer.transform(df["clean_text"])
y = df["news_category"]

_, X_test, _, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

predictions = model.predict(X_test)

print(classification_report(y_test, predictions))

# List the categories once so the matrix rows/columns are labeled.
labels = sorted(y.unique())

cm = confusion_matrix(y_test, predictions, labels=labels)
```

```
cm_table = pd.DataFrame(cm, index=labels, columns=labels)

print("\nConfusion Matrix (rows = actual, columns = predicted)")
print(cm_table)
```

Why: the classification report gives summary scores per category, but the confusion matrix shows the raw counts behind them — exactly which articles were classified correctly and which were confused for another category. Sorting the labels keeps the rows and columns in the same order so the diagonal (correct predictions) is easy to read.

Run:

```
python src/models/evaluate.py
```

Expected:

Why: Precision, recall, and F1-score show how well the model does for each category, not just overall.

```
precision
recall
f1-score
```

Reading the Confusion Matrix for Our Business Problem

A multi-class confusion matrix is a grid with one row and one column per category. The diagonal holds the correct predictions; everything off the diagonal is a mix-up. The example grid below shows the layout for four categories:

```
Confusion Matrix (rows = actual, columns = predicted)
      business  politics  sports  technology
business      310       18      2         25
politics       15      402      3         12
sports         1        4     540         6
technology     22        9      5        611
```

Read it row by row: each row is a true category, and the numbers across it show where those articles actually landed.

- **The diagonal** (business→business, sports→sports) is what you want high — these are correct.
- **Off-diagonal cells** are the confusions. Here business and technology trade a few articles (a story about a tech company's earnings could reasonably go either way), while sports is almost never confused with anything — its vocabulary is very distinct.

Not every mistake costs the same. Two categories that genuinely overlap (business vs technology) being occasionally swapped is tolerable; a breaking sports story landing in politics is worse because it reaches entirely the wrong feed. The summary metrics map onto this directly:

- **Recall on a category** — of all the articles that truly belong to it, how many did we catch? Low recall on, say, business means its stories are leaking into other feeds.
- **Precision on a category** — of everything we filed under it, how much truly belongs? Low precision means other categories' articles are polluting that feed.

A good news classifier keeps both high across every category — and pays special attention to the small categories, where a handful of mistakes swings the percentage sharply. Because the training data was balanced in Stage 6, the rare categories should already hold up here; this report is where you confirm it. Watch the per-category recall, find the weakest category, and read its row in the matrix to see exactly where any remaining mistakes are going.

Stage 9: Model Deployment (Hand-off)

Create:

***Why:** Wrapping the model in a FastAPI service lets other applications — a news app, a frontend, a content pipeline — request predictions over HTTP.*

```
src/api/app.py
```

***Why:** This defines a /predict endpoint that takes article text, cleans it with clean_text (the same step used in training), runs it through the saved vectorizer and model, and returns the category as JSON. It then calls route_article to file the article into that category's feed, so the search index and recommendation engine can pick it up. This endpoint is the hand-off the news app and recommendation layer will call later.*

```
# Lifecycle stage 9 – Model Deployment (hand-off)

from fastapi import FastAPI
from pydantic import BaseModel
import joblib
from src.data.preprocess import clean_text
from src.routing.store import route_article

app = FastAPI()

model = joblib.load("models/news_model.pkl")

vectorizer = joblib.load("models/tfidf_vectorizer.pkl")

class Article(BaseModel):
    text: str

@app.post("/predict")
def predict(article: Article):

    X = vectorizer.transform([clean_text(article.text)])

    category = model.predict(X)[0]
```

```
route_article(article.text, category)

return {"category": category}
```

Run:

Why: Starting the server makes the /predict endpoint live on your machine.

```
uvicorn src.api.app:app --reload
```

Open:

Why: FastAPI auto-generates interactive docs, so you can test the endpoint right in the browser without writing any client code.

```
http://127.0.0.1:8000/docs
```

Route Articles to Their Feed

Create:

Why: This is the “instant routing as news breaks” piece from the project goal. The moment an article is classified, this module files it into the matching category feed — the store the search index and recommendation engine read — instead of leaving it untagged.

```
src/routing/store.py
```

Why: It appends each classified article to a per-category CSV with a timestamp. A CSV keeps the local build simple; when you build the news app later, swap this for a database table or a message queue the recommendation engine consumes — only the storage call changes, not the pipeline.

```
# Lifecycle stage 9 – Category-feed routing

import csv
import os
from datetime import datetime

FEED_DIR = "data/feeds"

def route_article(text, category):

    os.makedirs(FEED_DIR, exist_ok=True)

    path = f"{FEED_DIR}/{category}.csv"

    new_file = not os.path.exists(path)

    with open(path, "a", newline="", encoding="utf-8") as f:
```



```
writer = csv.writer(f)

if new_file:
    writer.writerow(["timestamp", "category", "text"])

writer.writerow([datetime.now().isoformat(), category, text])
```

Note: Routing is what turns a bare prediction into something the rest of the platform can use. Each category feed is the seed the search index and recommendation engine will later read and rank — so a clean, consistent label here is what makes every downstream recommendation better.

Stage 10: CI/CD

The lifecycle does not end at deployment. CI/CD (Continuous Integration and Continuous Deployment) is what keeps the deployed model healthy over time, with as little manual work as possible.

For this project, a CI/CD pipeline would run automated tests on every code change (does the API still start? does cleaning still strip URLs and punctuation?), retrain the model when enough new articles have accumulated, re-check per-category precision and recall before letting a new model replace the old one, and redeploy the API and routing service automatically once the new model passes. Tools such as GitHub Actions, Docker, and a model registry are commonly used to wire these steps together.

This connects back to where we started in Stage 1. The news cycle constantly shifts — new topics, new vocabulary, even new categories — so a model trained once slowly goes stale, which is exactly the weakness of the fixed rule-lists we set out to replace. CI/CD turns the lifecycle you just worked through by hand into an automatic loop, so the classifier keeps learning new patterns from data and keeps routing new articles to the right feed without anyone re-running every stage.